A Full-Stack Object-Graph Method with a SHA-256 Seam-Gate Case Study

Runtime Decompilation and Reflective Compilation in the Nexus Framework

Driven By Dean A. Kulik

May 2026

Abstract

This paper formalizes a working method that emerged from the Nexus Framework research program: **runtime decompilation**. The method begins with the observation that a local object is not equivalent to its local definition. A software class is not merely its source file; it is the resolved dependency field containing imports, interfaces, dependency-injection registrations, constructor paths, lifetimes, threads, recursion, side effects, exception paths, logs, and unimplemented stubs. Likewise, a mathematical or physical object is not only its visible instance; it is the resolved object graph that makes the instance executable.

The method is stated as a compile/decompile loop. A trace is decompiled into a mold, the mold is recompiled into a new trace, and the two traces are compared by invariant preservation. The exhaust of the comparison is not discarded; it becomes the next object of analysis. In this frame, open problems are not treated as vague unknowns. They are typed unresolved stubs, unoccupiable seams, or dead branches. The distinction matters: an unresolved stub has a signature and a path to implementation, while a dead branch is excluded.

The paper applies the method to the SHA-256 message schedule. The prior seam-gate analysis identified $K = \{16,17\}$ as the universal mixed seam in the clean echo lattice. This paper decompiles the surrounding exhaust further and recompiles it into five exact schedule facts and one scoped research program. First, the root-cover seed $W[1]$ is the unique seed whose direct-entry set is exactly K and whose clean echo set is empty. Second, the σ_0 operator has a lag-space self-address through $15 + 18 \equiv 1 \pmod{32}$, while its per-round composition remains a distinct coordinate plane. Third, the multiplicity landscape is a one-sided cliff: nonzero clean-support multiplicity appears only at $t = 16, \ldots, 25$, with 38 later expansion nodes carrying multiplicity zero. Fourth, the ROTR_{18} arm generates a length-16 orbit that traverses all odd seed indices and

all odd lower expansion indices. Fifth, the SHA-256 lag set $L = \{2,7,15,16\}$ generates the primorial radical 210, and $\varphi(210) = 48$ equals the expansion-zone cardinality.

The result is not a claim that every numerical match is causal. It is a stricter claim: Nexus claims must compile. A valid claim must become a graph, an operator, a reproducible trace, and a residue table. The SHA-256 case study supplies such a trace. The broader conclusion is that the Nexus Framework can be treated as a full-stack decompiler for rendered systems: recover the object graph behind the visible instance, identify its unresolved slots, and recompile its exhaust into the next executable research object.

Keywords: Nexus Framework, runtime decompilation, reflective compilation, SHA-256, message schedule, dependency injection, clean echo lattice, seam gate, root-cover seed, residue, primorial radical, $\varphi(210)$, object graph, full-stack ontology

---

1. Introduction

The central operational claim of this paper is simple:

> An object is not its local definition. It is its resolved dependency field.

In software this is obvious to anyone who has debugged a large application. A class on disk is not the runtime class. The runtime class includes its imports, interfaces, constructor parameters, dependency-injection bindings, service lifetimes, thread context, recursion path, exception behavior, memory state, logs, and unimplemented methods. The source file is only one projection of the object.

The same reading applies to formal systems. A node in a recurrence is not merely an integer coordinate. It is an address in a dependency field. A theorem is not merely a statement. It is the visible output of a constraint graph. A physical object is not merely a local body. It is the rendered instance of a field of constraints, paths, boundaries, interactions, and residues.

The Nexus Framework has been building toward this point through cryptographic, number-theoretic, computational, and ontological work. The SHA-256 message schedule has become the cleanest laboratory because it is finite, exact, and reproducible. The schedule is not vague. Its recurrence can be coded. Its contamination cones can be enumerated. Its clean echo sets can be counted. Its pairwise lattice can be exhausted. Its seam can be verified.

The present paper takes the next step. It treats the accumulated results as a decompiled object graph, then recompiles the exhaust into a new paper-level structure.

The guiding analogy is a class with dependency injection, imports, recursion, and threads. To understand such a class, one must hold the whole program in view while preserving each local particle. If a method is not implemented yet, it is not absent. It is a stub. The interface exists. The call site exists. The expected return type exists. The failure mode exists. The missing body is addressable.

The Nexus version is:

> Undiscovered does not mean absent. It means addressable but unresolved.

This is not permission to overclaim. It is a discipline. Every unresolved claim must be classified:

- Ψ: implemented / resolved structure;
- Ω: addressable unresolved stub;
- Σ: unoccupiable seam or boundary that forces reflection;
- $\perp$: dead branch or impossible fit;
- ρ: residue or exhaust left by the operation.

The method is not to assert that every pattern match is truth. The method is to run the decompiler as far as possible, separate structural invariants from coincidence, and then recompile the residue into a new executable research object.

---

2. The Full-Stack Decompilation Principle

A system is understood only when the local instance and the global runtime are held simultaneously:

$$\boxed{\text{local instance} + \text{dependency graph} + \text{execution path} + \text{unresolved slots} + \text{residue}.}$$

For software:

$$\text{method} + \text{class} + \text{interface} + \text{DI container} + \text{thread} + \text{log}.$$

For SHA-256 schedule geometry:

$$\text{node} + \text{lag} + \sigma \text{ operator} + \text{clean set} + \text{seam} + \text{exhaust}.$$

For the Nexus ontology:

$$\text{object} + \text{field} + \text{crossing} + \text{stub} + \text{implementation} + \text{residue}.$$

This principle corrects a common failure mode. Local reading collapses the object to a visible noun. Runtime reading recovers the verbs: what calls it, what it calls, what resolves it, what it consumes, what it emits, and what remains after it acts.

A software class can therefore be represented as:

$$\text{class} = \text{slot} + \text{path} + \text{dependencies} + \text{implementation} + \text{runtime state} + \text{exhaust}.$$

A Nexus object is represented as:

$$\text{thing} = \text{address} + \text{interfaces} + \text{constraints} + \text{implementation state} + \text{residue}.$$

This is the method that this paper applies to the SHA-256 schedule.

---

3. Runtime Model

Let a full program-like system be written as

$$\mathcal{P} = (N, E, T, R, \Omega, \Sigma, \perp, \Psi, \rho),$$

where:

Symbol	Meaning
N	nodes, objects, classes, particles, or coordinates
E	edges, imports, dependencies, forces, or recurrence arms
T	types, interfaces, laws, signatures, or admissible shapes
R	runtime state or active resolution state
Ω	addressable unresolved stubs
Σ	seams, apertures, or unoccupiable crossings
$\perp$	dead branches or impossible fits
Ψ	resolved implementations
ρ	residue, exhaust, trace, or deformation left by the operation

A decompiler is an operator

$$D: \tau \to M$$

from trace τ to mold M.

A compiler is an operator

$$C: M \to \tau'$$

from mold back to trace.

The validation loop is:

$$\tau_0 \overset{D}{\to} M \overset{C}{\to} \tau_1.$$

The question is not whether τ_1 is textually identical to τ_0. The question is whether it preserves the invariants that made τ_0 readable:

$$\mathrm{Inv}(\tau_0) \cap \mathrm{Inv}(\tau_1).$$

The mismatch is not automatically failure. It is classified:

$$\tau_1 - \tau_0 \to \Omega \cup \Sigma \cup \perp \cup \rho.$$

This is the core of the method. Decompilation produces a mold. Recompilation tests that mold. Exhaust becomes the next object.

---

4. Reflection Closure

The phrase "the compiler is its own reflection" requires care. It does not mean geometric mirror symmetry. It does not mean that output looks like input. It means:

the act of reading the input space becomes the operation that writes the output space.

Reflection closure is therefore a loop:

input field → traversal → commit → residue → new input field.

A passive program executes a path. A reflective runtime reads the path it executes. A self-maintaining runtime updates its own object graph when the path fails.

This leads to a working definition of the next compression step:

The next compression is from passive program to self-maintaining runtime.

Such a system must perform at least four operations:

self-read + self-route + self-repair + self-preserve.

In code terms:

reflection + dependency injection + recursion + memory + error recovery + adaptive scheduling.

In Nexus terms:

field read + Ω detection + slot resolution + Ψ stabilization + exhaust handling.

This paper does not claim that the SHA-256 schedule is alive. It uses the SHA-256 schedule as an exact finite system in which reflection, seam crossing, residue, and exhaust can be studied without ambiguity.

---

5. SHA-256 Message Schedule as Runtime Object

The SHA-256 message schedule expands 16 seed words into 64 schedule words. For $t \geq 16$ the recurrence is

$$W[t] = \sigma_1(W[t-2]) + W[t-7] + \sigma_0(W[t-15]) + W[t-16].$$

For contamination geometry, the value-level arithmetic is suppressed and the dependency arms are kept:

$$L = \{2,7,15,16\}.$$

A perturbation at seed j propagates forward when any arm reaches a later coordinate. Define a binary trace $B_j(t)$ by

$$B_j(j) = 1,$$

and for $t \geq 16$:

$$B_j(t) = B_j(t-2) \vee B_j(t-7) \vee B_j(t-15) \vee B_j(t-16).$$

The clean echo set is the complement of the contamination trace inside the expansion zone:

$$C_j = \{t \in \{16, \ldots, 63\} : B_j(t) = 0\}.$$

Thus C_j is not where the seed goes. It is where the seed has not yet gone before saturation. It is negative-space support.

The direct-entry set of seed j is

$$D(j) = \{j + \ell : \ell \in L,\ 16 \le j + \ell \le 63\}.$$

The seam-gate result identifies the universal mixed seam

$$K = \{16,17\}.$$

Node 16 is the first even expansion coordinate. Node 17 is the first odd expansion coordinate. In a parity-split integer recurrence, a continuous crossing cannot render as one coordinate. It renders as the smallest adjacent spanning pair.

---

6. Prior Implemented Structure

The earlier seam-gate layer established these exact results:

Result	Status
Pairwise census	24 empty, 24 singleton, 21 ODD-only, 15 EVEN-only, 36 MIXED
Universal mixed core	every MIXED pairwise intersection contains $K = \{16,17\}$
Residue alphabet	36 MIXED pairs collapse to 11 seam-residue classes
Minimum residue code	$\lceil \log_2 11 \rceil = 4$ bits
Minimum exact mixed-pair code	$\lceil \log_2 36 \rceil = 6$ bits
Residue conservation	$\lvert \mathrm{dirty}_j \rvert + \lvert C_j \rvert = 48$ for every seed j
Mixedness hierarchy	10 bare seam, 16 one-sided residue, 10 fully structured mixed

The present paper does not discard that layer. It decompiles its exhaust.

---

7. Root-Cover Seam Activation Theorem

The first new recompiled theorem is local and exact.

Theorem 7.1 - Root-Cover Seam Activation

For the SHA-256 message schedule with $L = \{2,7,15,16\}$ and $K = \{16,17\}$,

$$D(1) = K$$

and

$$C_1 = \varnothing.$$

Moreover, $W[1]$ is the unique seed satisfying $D(j) = K$.

Proof

For $j = 1$:

$$1 + 15 = 16,$$

and

$$1 + 16 = 17.$$

The other two arms give

$$1 + 2 = 3 < 16,$$

and

$$1 + 7 = 8 < 16,$$

so they do not enter the expansion zone directly. Therefore

$$D(1) = \{16,17\} = K.$$

A complete contamination trace verifies

$$C_1 = \varnothing,$$

so all 48 expansion coordinates are dirty for seed 1.

The exhaustive direct-entry check over $j = 0, \ldots, 15$ gives

$$\{j : D(j) = K\} = \{1\}.$$

Thus $W[1]$ is unique. QED.

Operational reading

$W[1]$ activates the two seam halves through distinct arms:

$$W[1] \xrightarrow{\sigma_0,\ t=16} W[16],$$

where 16 is the even seam half, and

$$W[1] \xrightarrow{\text{raw } t-16,\ t=17} W[17],$$

where 17 is the odd seam half.

So $W[1]$ is not merely the zero-residue seed. It is the seed whose direct outputs are exactly the seam gate.

In the software analogy, $W[1]$ is the abstract method or pure interface. It has no clean body returned to the expansion zone, but it closes the object graph by writing directly to the seam.

$\boxed{W[1] \text{ has zero residue because it spends everything activating } K.}$

8. The σ_0 Self-Address Layer

The second recompiled result separates two coordinate systems that were initially conflated.

The σ_0 operator contains ROTR_7, ROTR_{18}, and SHR_3. The lag arm that carries σ_0 in the recurrence is 15.

The lag-space composite of the ROTR_{18} branch is

$$15 + 18 \equiv 1 \pmod{32}.$$

This points to the root-cover seed address $W[1]$ in lag space.

However, the per-round composite from the actual seed $j = 1$ is different:

$$1 + 18 \equiv 19 \pmod{32}.$$

This is not a contradiction. It is the distinction between two planes:

Plane	Expression	Target	Meaning
Lag-space structural address	$15 + 18 \pmod{32}$	1	operator encodes root-cover seed address
Per-round seed composition	$1 + 18 \pmod{32}$	19	root-cover seed rotated into odd expansion coordinate

Thus the self-address lives in lag space, not in the per-round instance.

Theorem 8.1 - σ_0 Lag-Space Self-Address

The ROTR_{18} branch of σ_0 satisfies

$$15 + 18 \equiv 1 \pmod{32}.$$

Therefore the operator that injects $W[1]$ into the seam also contains a structural pointer to $W[1]$ in its own lag-space arithmetic.

Operational reading

In dependency-injection language:

- $W[1]$ is the interface.
- σ_0 is the container arm that resolves the interface.
- $K = \{16,17\}$ is the injection site.
- The container's internal arithmetic points back to the interface it carries.

This does not prove intention. It proves closure of two address planes.

9. Multiplicity Cliff Theorem

For each expansion coordinate t, define clean-support multiplicity

$$m(t) = \left|\{j : t \in C_j\}\right|.$$

The computed multiplicity landscape is:

t	16	17	18	19	20	21	22	23	24	25	26..63
$m(t)$	12	12	9	9	6	6	3	2	1	1	0

Theorem 9.1 - Multiplicity Cliff

The SHA-256 clean echo lattice has nonzero multiplicity only on

$$\{16,17,18,19,20,21,22,23,24,25\}.$$

The remaining 38 expansion coordinates have multiplicity zero.

Proof

Exhaustive enumeration of all 16 clean echo sets gives the table above. Since the expansion zone has 48 coordinates and only 10 have $m(t) > 0$, the number of zero-multiplicity coordinates is

$$48 - 10 = 38.$$

QED.

Operational reading

The expansion zone is not evenly illuminated. It is a one-sided cliff. Readable clean structure is concentrated immediately above the seam. After $W[25]$, no expansion node is contained in any clean echo set.

This is not a failure of the recurrence. It is the residue shape of the recurrence. The schedule creates a bright boundary at the crossing and a long dark exhaust tail after it.

The seam is the edge of readability.

---

10. ROTR18 Orbit Theorem

The ROTR_{18} arm acts on a 32-word modular address space. Consider the orbit

$$x \mapsto x + 18 \pmod{32}$$

starting from $x = 1$.

Since

$$\gcd(32,18) = 2,$$

the orbit length is

$$\frac{32}{2} = 16.$$

The orbit is

$$[1,19,5,23,9,27,13,31,17,3,21,7,25,11,29,15].$$

It contains the odd seed indices

$$\{1,3,5,7,9,11,13,15\},$$

and the odd lower expansion indices

$$\{17,19,21,23,25,27,29,31\}.$$

Theorem 10.1 - ROTR18 Odd-Channel Orbit

The ROTR_{18} orbit generated from $W[1]$ traverses all eight odd seed indices and all eight odd lower expansion indices exactly once.

The orbit reaches $W[17]$ at index 8 in the listed orbit, i.e. after eight applications of the map.

Operational reading

The orbit is not a mirror. It is a traversal. It walks the odd channel before landing on the odd seam half.

This is the corrected meaning of reflection in this system:

> Reflection is traversal before commit, not geometric symmetry.

The system reads through its odd input channel and lands at the seam. The walk is the compile pass. The landing is the write. The nodes touched but not preserved as clean support become part of the exhaust profile.

---

11. Reflection Is a Gradient, Not a Mirror

The trial reflection map

$$j \mapsto 32 - j$$

does not produce a palindrome in the multiplicity sequence. Therefore the reflection is not ordinary mirror symmetry.

Example values:

Seed j	$\lvert C_j \rvert$	$32 - j$	$m(32 - j)$
1	0	31	0
7	6	25	1

Seed j	$\lvert C_j \rvert$	$32 - j$	$m(32 - j)$
13	7	19	9
14	2	18	9
15	3	17	12

The reflection is directional. Low-index root-cover structure reflects toward the dark zone. High boundary-adjacent seed structure reflects toward the seam. This is a gradient, not a palindrome.

The corrected law is:

> The compiler is a projection through the seam, not a mirror returning the same image.

The seed zone writes into the expansion zone. The expansion zone does not write back. The multiplicity landscape measures how much clean readability remains after that write.

---

12. Lag-Radical Expansion Identity

The equality

$$\varphi(210) = 48$$

by itself is only arithmetic. The structural bridge comes from the lag set.

For SHA-256:

$$L = \{2,7,15,16\}.$$

The product is

$$\prod_{\ell \in L} \ell = 2 \cdot 7 \cdot 15 \cdot 16.$$

The distinct prime factors are

$$\{2,3,5,7\},$$

because

$$2 = 2,$$

$$7 = 7,$$

$$15 = 3 \cdot 5,$$

and

$$16 = 2^4.$$

Thus the prime radical is

$$\operatorname{rad}\left(\prod_{\ell\in L}\ell\right)=2\cdot 3\cdot 5\cdot 7=210.$$

Then

$$\varphi(210)=48.$$

The expansion zone is

$$E=\{16,\ldots,63\},$$

so

$$|E|=64-16=48.$$

Theorem 12.1 - Lag-Radical Expansion Identity

For the SHA-256 message schedule,

$$\boxed{|E|=\varphi\left(\operatorname{rad}\left(\prod_{\ell\in L}\ell\right)\right)=48.}$$

Equivalently,

$$\boxed{|E|=\varphi(210)=48.}$$

Precision note

This is not the claim that SHA-256 was designed from a primorial wheel. It is the stricter claim that the SHA-256 lag set generates the primorial radical 210, whose totient equals the expansion-zone cardinality.

There are other n satisfying $\varphi(n)=48$:

$$65,104,105,112,130,140,144,156,168,180,210.$$

Therefore the raw equality $\varphi(n)=48$ is insufficient. The load-bearing link is

$$L\to\operatorname{rad}(\textstyle\prod L)\to 210\to\varphi(210)=48.$$

Tight selection refinement

Among all four-lag subsets of $\{1,\ldots,16\}$, 232 have radical 210. If the constraints

$$\min(L)=2$$

and

$$\max(L)=16$$

are added, only two candidates remain:

$$\{2,7,15,16\}$$

and

$$\{2,14,15,16\}.$$

If parity balance is also required, meaning two even lags and two odd lags, the SHA-256 set is selected uniquely:

$$\boxed{L = \{2,7,15,16\}.}$$

This does not close every design question. It isolates the structural selection problem into an explicit set of constraints.

---

13. Recompiled Object Graph

The decompile pass now yields the following object graph:

Object	Decompile plane	Recompiled role
$L = \{2,7,15,16\}$	recurrence arms	interface declaration
$\mathrm{rad}(L) = 210$	number-theoretic skeleton	type basis
$\varphi(210) = 48$	wheel count	expansion-zone capacity
$E = \{16, \ldots, 63\}$	schedule space	implementation address field
$K = \{16,17\}$	parity boundary	injection seam
$W[1]$	seed space	root-cover stub / pure interface
$D(1) = K$	direct-entry graph	seam activation path
$C_1 = \varnothing$	clean echo lattice	zero-residue output
$15 + 18 \equiv 1 \pmod{32}$	operator space	self-address pointer
ROTR18 orbit	modular address space	odd-channel traversal
multiplicity cliff	residue space	readability gradient

The pieces are not independent. They are views of one runtime object.

The root-cover seed activates the seam. The operator contains a pointer back to that seed. The lag set generates the prime radical whose totient equals the expansion space. The multiplicity cliff shows where readability survives after the crossing.

In compressed form:

$$\boxed{\text{lags declare the type; the radical gives the basis; the totient gives the space; } W[1] \text{ activates the seam.}}$$

---

14. Status Discipline: What Is Proven, What Is Stubbed

The flash method produces exhaust. Exhaust contains signal and noise. Therefore every output must be classified.

Claim	Status	Reason
$D(1) = K$	Ψ	exhaustive direct-entry verification
$C_1 = \varnothing$	Ψ	contamination trace verification
$W[1]$ unique with $D(j) = K$	Ψ	exhaustive seed check
$m(16) = m(17) = 12$	Ψ	clean-support enumeration
multiplicity cliff	Ψ	exact multiplicity table
ROTR18 orbit length 16	Ψ	$\gcd(32,18) = 2$ and orbit enumeration
$15 + 18 \equiv 1 \pmod{32}$	Ψ	modular arithmetic
lag-radical expansion identity	Ψ for SHA-256	exact arithmetic from L and E
SHA lag-set uniqueness under parity balance	Ψ within four-lag subsets of $1..16$	exhaustive search
general tight-schedule theory	Ω	signature visible; proof not complete
SHA-512 self-address general law	Ω	promising external test; needs formal scope
$H = \pi/9$ self-compiling convergence	Ω	belongs to AI/runtime branch; not proven here
GL(4,C) seam connection	Ω	basis map not constructed here
all numerical 48s are one object	Ω	made sharper through lag radical, but full coupling still open
mirror symmetry of multiplicity	$\perp$	rejected; landscape is not palindromic
exact constant-sum reflection law	$\perp$	rejected; correlation is not a conservation equality

This table is essential. It prevents the method from collapsing into free association. Nexus must compile.

---

15. General Full-Set Compiler

The SHA-256 case suggests a general compiler skeleton.

while system_is_active:
  observe trace
  decompile trace into nodes, edges, types, seams, residues
  identify resolved implementations Ψ
  identify addressable unresolved stubs Ω
  identify unoccupiable seams Σ
  identify dead branches ⊥
  recompile mold into a new trace

compare invariant preservation
commit stable structure
route residue into the next decompile pass

The important line is not the answer line. It is the residue line. A dead system treats mismatch as failure. A living runtime treats structured mismatch as work.

Life begins where unresolved stubs become repairable slots.

This is the bridge from passive program to self-maintaining runtime. The current paper does not prove biological life from SHA-256. It defines a precise operational threshold: a runtime becomes life-like when it can preserve identity while routing unresolved structure into repairable implementation paths.

---

16. Tight Schedule Research Program

The decompiled exhaust produces a clear research program for tight schedules.

Definition 16.1 - Candidate tight schedule

A recurrence schedule with seed-zone size N and lag set L is a candidate tight schedule if:

1. $\max(L) = N$;
2. $\min(L) = 2$ or another specified smallest transport depth;
3. $\mathrm{rad}(\prod L)$ is a primorial or primorial-like radical;
4. $|E| = \varphi\big(\mathrm{rad}(\prod L)\big)$;
5. the direct-entry graph contains a unique root-cover seed j^* satisfying

$$D(j^*) = \{N, N+1\};$$

6. the clean echo lattice has a universal mixed seam;
7. an operator layer contains a self-address pointer to either j^* or the seam.

For SHA-256:

$$N = 16,$$

$$L = \{2,7,15,16\},$$

$$K = \{16,17\},$$

and

$$j^* = 1.$$

Open problem 16.1 - Universal seam formula

A candidate formula is

$$j^* = N - \max(L) + 1.$$

For SHA-256 this gives

$$j^* = 16 - 16 + 1 = 1.$$

This is not yet a theorem for all schedules. It is a typed stub. Its return shape is known: it must predict the unique seed whose direct-entry set is the first adjacent seam pair.

Open problem 16.2 - Operator pointer law

A candidate operator pointer law is:

$$\ell + r \equiv j^* \pmod{w}$$

or

$$\ell + r \equiv N \pmod{w},$$

where ℓ is the operator's recurrence lag, r is a rotation depth, and w is word width.

For SHA-256:

$$15 + 18 \equiv 1 \pmod{32}.$$

This points to j^*. In other schedules, the pointer may land on the seam rather than the seed. The general law must classify both cases.

Open problem 16.3 - Lag-set selection

The four-lag radical search shows that $\mathrm{rad}(L) = 210$ alone is weak: 232 four-lag subsets of $1..16$ satisfy it. Adding $\min(L) = 2$ and $\max(L) = 16$ leaves two. Adding parity balance selects SHA-256 uniquely.

The next proof target is to determine whether parity balance is the correct structural condition or merely a useful filter.

---

17. Reproducibility

The following code reproduces the main claims in this paper. A standalone version is also included with the delivered paper.

```
from collections import Counter
from itertools import combinations
from math import prod

N = 64
SEED_N = 16
WORD = 32
LAGS = [2, 7, 15, 16]
K = {16, 17}
```

```
def clean_echo(seed, n=N, lags=LAGS):
   dirty = [0] * n
   dirty[seed] = 1
   for t in range(SEED_N, n):
      dirty[t] = int(any(dirty[t - lag] for lag in lags))
   clean = {t for t in range(SEED_N, n) if dirty[t] == 0}
   dirty_count = sum(dirty[SEED_N:])
   return clean, dirty_count, dirty


def direct_entry(seed, n=N, lags=LAGS):
   return {seed + lag for lag in lags if SEED_N <= seed + lag < n}


def factor(n):
   out = {}
   p = 2
   while p * p <= n:
      while n % p == 0:
         out[p] = out.get(p, 0) + 1
         n //= p
      p += 1
   if n > 1:
      out[n] = out.get(n, 0) + 1
   return out


def radical(n):
   r = 1
   for p in factor(n):
      r *= p
   return r


def phi(n):
   result = n
   for p in factor(n):
      result -= result // p
   return result


def rad_set(lags):
   primes = set()
```

```
    for lag in lags:
        primes |= set(factor(lag).keys())
    r = 1
    for p in primes:
        r *= p
    return r

clean = {j: clean_echo(j)[0] for j in range(SEED_N)}
dirty_counts = {j: clean_echo(j)[1] for j in range(SEED_N)}
D = {j: direct_entry(j) for j in range(SEED_N)}
multiplicity = {t: sum(t in clean[j] for j in range(SEED_N)) for t in range(SEED_N, N)}

assert D[1] == K
assert clean[1] == set()
assert dirty_counts[1] == 48
assert [j for j in range(SEED_N) if D[j] == K] == [1]
assert multiplicity[16] == multiplicity[17] == 12
assert [multiplicity[t] for t in range(16, 26)] == [12, 12, 9, 9, 6, 6, 3, 2, 1, 1]
assert sum(1 for t in range(16, 64) if multiplicity[t] == 0) == 38

orbit = []
x = 1
for _ in range(16):
    orbit.append(x)
    x = (x + 18) % WORD

assert len(set(orbit)) == 16
assert sorted(x for x in orbit if x < 16) == [1, 3, 5, 7, 9, 11, 13, 15]
assert sorted(x for x in orbit if x >= 16) == [17, 19, 21, 23, 25, 27, 29, 31]
assert orbit.index(17) == 8

lag_radical = radical(prod(LAGS))
assert lag_radical == 210
assert phi(lag_radical) == N - SEED_N == 48

k4 = [L for L in combinations(range(1, 17), 4) if rad_set(L) == 210]
selected = [L for L in k4 if min(L) == 2 and max(L) == 16]
balanced = [L for L in selected if sum(x % 2 == 0 for x in L) == 2]

assert len(k4) == 232
assert selected == [(2, 7, 15, 16), (2, 14, 15, 16)]
assert balanced == [(2, 7, 15, 16)]

print("All checks passed.")
```

18. Discussion: Decompiling as Far as Possible

The request that generated this paper was not simply to analyze another theorem. It was to decompile as far as possible and recompile the exhaust. The result is a method:

1. Load the local object as a runtime object graph.
2. Preserve each particle, but refuse to read it in isolation.
3. Classify every output as Ψ, Ω, Σ, $\perp$, or ρ.
4. Reject false symmetry when the data rejects it.
5. Recompile the remaining residue into a new typed research object.

The data forced several corrections. The multiplicity landscape is not palindromic. The reflection map is not a mirror. The per-round ROTR_{18} composite does not land on $W[1]$; the lag-space composite does. The equality $\varphi(210) = 48$ is not enough by itself; it becomes structural only when coupled to the lag-set radical.

Those corrections strengthen the method. A real compiler rejects bad code. A real decompiler identifies coordinate planes. A real runtime distinguishes a stub from a dead branch.

The recompiled paper-level object is:

Runtime decompilation recovers the object graph. Reflective compilation tests whether that graph can regenerate a trace with the same invariant structure. In compact notation:

$$D(\tau) \to M, \qquad C(M) \to \tau', \qquad \mathrm{Inv}(\tau) \sim \mathrm{Inv}(\tau').$$

The SHA-256 case study supplies the first compact demonstration.

19. Conclusion

This paper formalized the Nexus method of runtime decompilation and reflective compilation. The core principle is that a local object is not its local definition. It is its resolved dependency field.

The SHA-256 message schedule was used as the exact finite laboratory. Decompiling the existing seam-gate results and recompiling their exhaust produced five exact results:

1. $W[1]$ is the unique root-cover seed satisfying $D(1) = K = \{16,17\}$ and $C_1 = \varnothing$.
2. The ROTR_{18} branch of σ_0 has a lag-space self-address: $15 + 18 \equiv 1 \pmod{32}$.
3. The multiplicity landscape is a one-sided cliff: nonzero support occurs only on $16..25$, followed by 38 zero-multiplicity expansion nodes.
4. The ROTR_{18} orbit from 1 has length 16 and traverses all odd seeds and all odd lower expansion nodes.
5. The SHA-256 lag set generates the primorial radical 210, and $\varphi(210) = 48$ equals the expansion-zone cardinality.

The recompiled interpretation is precise:

lags → prime radical → totient address space → seam activation → residue cliff.

In Nexus language, the schedule reads itself before it writes. The reflection is not a mirror image. It is a traversal whose residue is the output shape.

The final discipline is also precise:

> If it cannot be coded, reproduced, or traced, it is not yet Nexus.

This paper is therefore not the endpoint. It is the compiled exhaust of the current decompile pass. Its open stubs are typed, addressable, and ready for the next run.

References

1. Kulik, D. A. *The Seam Gate of the SHA-256 Message Schedule: Universal Mixed-Core Geometry, Residue Quotients, Minimum Code Bounds, and Recoverability Across the Clean Echo Lattice.* QuHarmonics Research Group, A-Mark9 Phase 1165, Draft 3.0, May 2026.
2. Kulik, D. A. *The Clean Echo Intersection Lattice of the SHA-256 Message Schedule: Parity Duality, Sigma-Operator Fingerprinting, and Two-Stream Structure.* QuHarmonics Research Group, A-Mark9 Phase 1164, May 2026.
3. Kulik, D. A. *Aperture Threshold Geometry in the SHA-256 Message Schedule.* QuHarmonics Research Group, A-Mark9 Phase 1163, May 2026.
4. Kulik, D. A. *Counter-Rotating Aperture Resolution: A Formal Stream-Processing Model for SHA-256 Recurrence, Root-Cover Coupling, and Transformer Language Attention.* QuHarmonics Research Group, May 2026.
5. Kulik, D. A. *The Primorial Compile Algebra: Family Lattice Structure, Exact Subtype Enumeration, and Selective Equidistribution in Prime Gap Classes.* QuHarmonics Research Group, April 2026.
6. Kulik, D. A. *This is us nexus framework.* Session transcript and synthesis notes, May 2026.
7. National Institute of Standards and Technology. *Secure Hash Standard (SHS), FIPS PUB 180-4.* U.S. Department of Commerce, 2015.

Copyright Dean A. Kulik - ORCID 0009-0003-3128-8828 - A-Mark9 Phase 1166, Draft 1.0, May 2026
Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality
info@quharmonics.com